

Base Dados II

2º Trabalho Prático: Trabalho 2: PL/SQL E APEX

GRUPO 11

Gestão de uma Estância Termal

Integrantes do grupo:

a43720 - Liliana Paquete

a43781 - Natalino Gomes

47380 - Toni Lopes

Conteúdo

INTRODUÇÃO	3
DESENVOLVIMENTO.....	4
1. Modelo Relacional.....	4
2. Normalização.....	5
3. PLSQL UTILIZADOS e Aplicação APEX.....	6
3.1 Procedimentos.....	6
3.2 Funções.....	49
3.4 Triggers.....	51
3.5 Packages.....	56
Conclusão	62

INTRODUÇÃO

Mediante a implementação do nosso trabalho que consiste em implementar uma base dos dados para fazer a gestão de uma Estância Termal, após a elaboração do primeiro trabalho nos vimos na necessidade de alterar algumas informações e tendo em conta este dilema chegou-se ao consenso de que o modelo iria possuir as seguintes entidades:

- Serviço
- Funcionário
- Cliente
- Fornecedor
- Produto
- Departamento
- Chefe

O Funcionário realiza Serviços, na Estância mediante o seu cargo.

O Cliente sendo a pessoa que faz a requisição dos Serviços pretendido.

O departamento onde cada determinado funcionário vai trabalhar.

O Chefe que supervisiona os funcionários do seu determinado departamento.

O Fornecedor que fornece os produtos necessários para o bom funcionamento da Terma

E o Produto que é fornecido pelos fornecedores para a realização dos serviços e etc...

DESENVOLVIMENTO

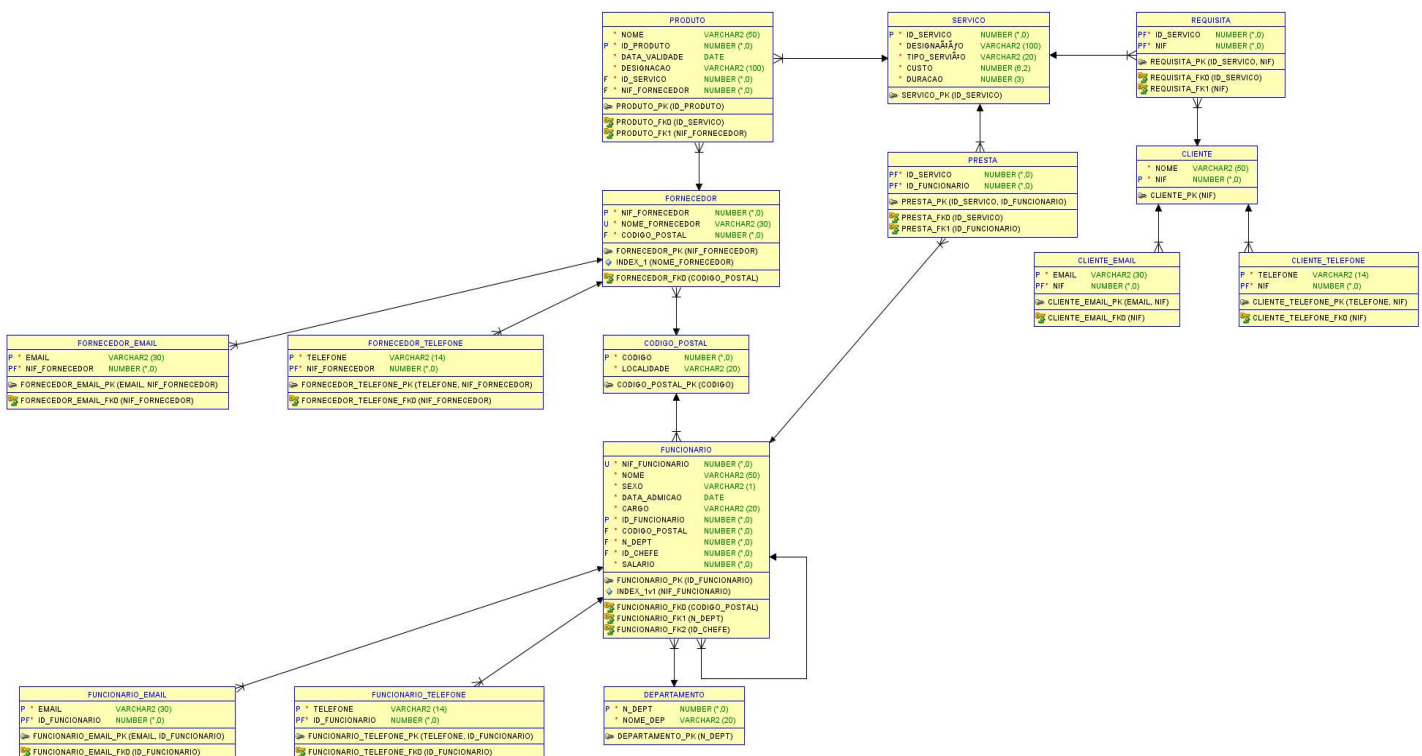
1. MODELO RELACIONAL

O modelo Relacional por consequência também sofreu algumas alterações que são

Para os campos telefones e os emails foram removidas da sua entidade principal e colocadas em tabela separadas, ligadas a tabela principal com uma chave primaria da tabela principal, a chave juntamente com o respetivo campo telefone ou email forma-se assim uma chave composta por esses 2 atributos respeitando assim a primeira forma normal (onde os atributos têm de ser atômicos).

Para a segunda forma normal, os atributos não chave como código postal e localidade que não dependem apenas da chave das respetivas tabela em que se encontram são colocados numa tabela separada, pois um depende do outro, fazendo isso já nos encontramos na 3ª forma visto que não já não temos nenhum atributo nas tabelas dependentes um do outro pois na terceira forma normal é necessário que todos os atributos das tabelas sejam funcionalmente independentes uns dos outros, ao mesmo tempo, dependentes exclusivamente da chave primária da tabela.

Nas tabelas principais foi colocado chave estrangeira das respetivas tabelas criadas.



2. NORMALIZAÇÃO

Relações Obtidas a Partir do Modelo E-R:

Os esquemas relacionais normalizados das relações obtidas a partir do modelo E-R são os seguintes:

- Codigo_postal (codigo(PK), Localidade)
- Departamento (N_Dept(PK), Nome_Dept)
- Fornecedor (NIF_Fornecedor(PK), nome, codigo_postal (FK))
- Funcionario (ID_Funcionario (PK), NIF_Funcionario, nome, sexo, data_admicao, cargo, código_postal (FK), N_Dept (FK), ID_Shefe(FK), salario)
- Servico (ID_servico(PK), tipo_servico, custo, designacao, duracao)
- Cliente (NIF(PK), nome)
- Produto (ID_Produto(PK), nome, Designacao, data_validade, id_servico(FK), Nif_Fornecedor(FK))
- Presta (ID_servico(PK)(FK), ID_Funcionario (PK)(FK))
- Requisita (id_servico(FK)(PK), Nif_Cliente(FK)(PK))
- Funcionario_telefone(ID_funcionario (FK)(PK), telefone(PK))
- Funcionario_email(ID_funcionario (FK)(PK), email(PK))
- Fornecedor_email(NIF_Fornecedor(FK)(PK), email(PK))
- Fornecedor_telefone(NIF_Fornecedor(FK)(PK), telefone(PK))
- Cliente_email(NIF_Cliente(FK)(PK), email(PK))
- Cliente_telefone(NIF_Cliente(FK)(PK), telefone(PK))

3. PLSQL UTILIZADOS e Aplicação APEX

Procedimentos

TRAB_1

Código Plsql

```
create or replace PROCEDURE TRAB_1(id_depart IN NUMBER) AS
dept_id NUMBER;
avg_sal NUMBER;
min_sal NUMBER;
max_sal NUMBER;
invalid_id_depart EXCEPTION;
BEGIN

dept_id := id_depart;
IF id_depart <= 0 THEN
    RAISE invalid_id_depart;
else
SELECT AVG(salario), MIN(salario), MAX(salario) INTO avg_sal, min_sal, max_sal
FROM Funcionario
WHERE Funcionario.id_dept = dept_id;

http.p ('Numero do departamento: ' || to_char(id_depart) || '<br/>');
http.p ('A média de salarios do departamento é: ' || to_char(avg_sal) || '<br/>');
http.p ('O salário mínimo da departamento é: ' || to_char(min_sal) || '<br/>');
http.p ('O salário máximo do departamento é: ' || to_char(max_sal) || '<br/>');
end if;

EXCEPTION
WHEN invalid_id_depart THEN
    dbms_output.put_line('ID must be greater than zero!');
WHEN NO_DATA_FOUND THEN
    HTP.P('Departamento nº ' || TO_CHAR(id_depart) || ': Não existe');
END TRAB_1;
```

Ouput

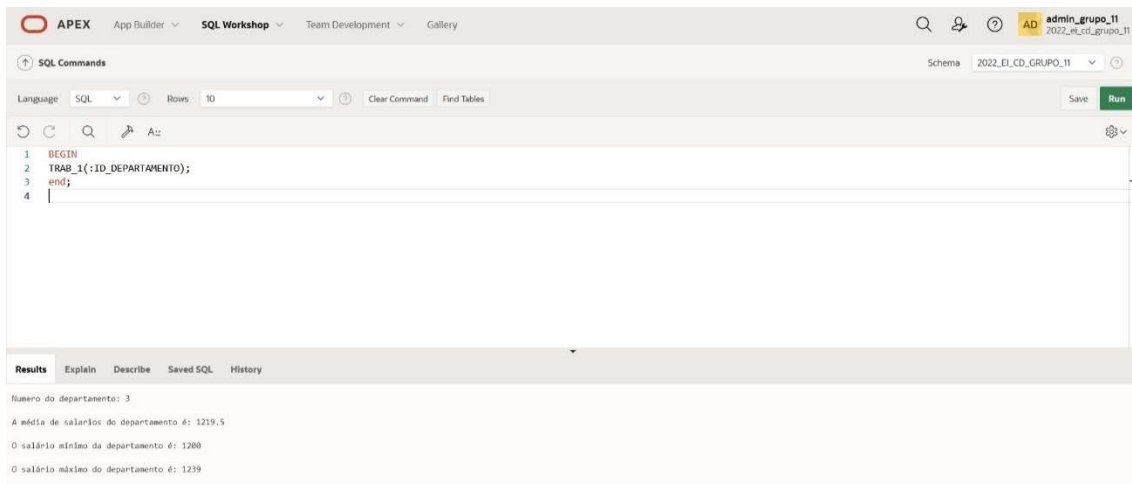


Figura 1: trab_1

Apex

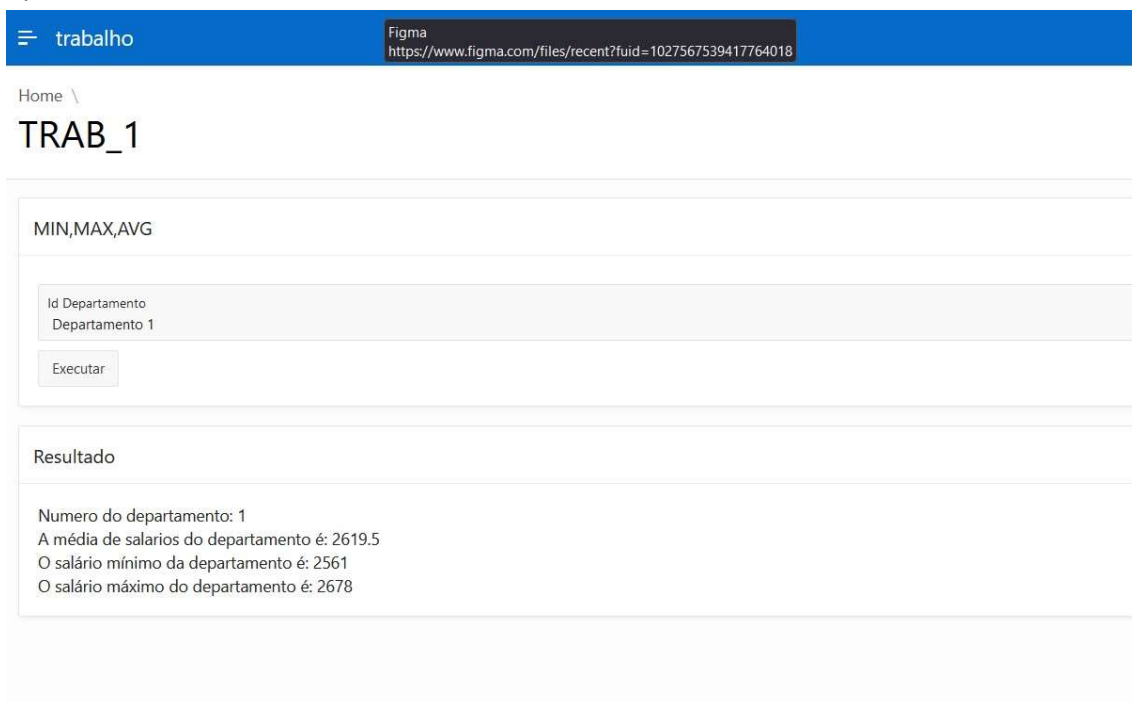


Figura 2: Resultado Apex_1

TRAB_2_1

Código PLSQL

```
create or replace PROCEDURE TRAB_2_1(NIF_F in DEPARTAMENTO.id_dept%type ) AS
  ID_F funcionario.ID_FUNCIONARIO%type;
  NOME_f funcionario.NOME%type;
  NOME_D DEPARTAMENTO.NOME_DEP%type;

BEGIN
  SELECT ID_FUNCIONARIO, NOME, NOME_DEP
  into ID_F, NOME_f, NOME_D
  FROM funcionario
  left join DEPARTAMENTO on funcionario.id_dept=DEPARTAMENTO.id_dept
  where funcionario.NIF_FUNCIONARIO= NIF_F;
  http.p('ID_Funcionar: ' || ID_F || '<br />');
  http.p('Nome Funcionario: ' || NOME_f || '<br />');
  http.p('Nome Departamento: ' || NOME_D || '<br />');

EXCEPTION

  WHEN NO_DATA_FOUND THEN
    http.p ('Nif não existe!' || '<br />');
  WHEN others THEN
    http.p('Error!' || '<br />');
END TRAB_2_1;
```

Output

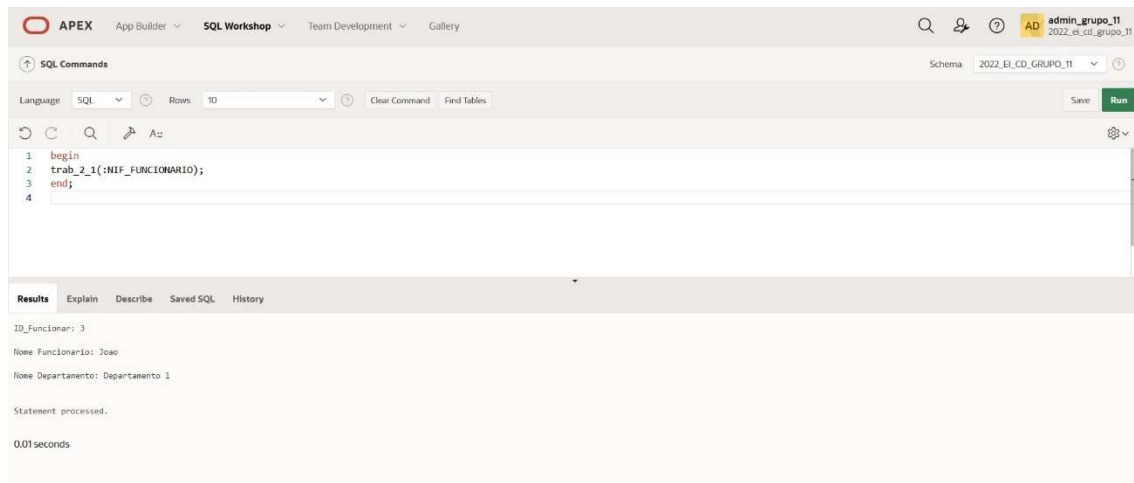


Figura 3: trab_2_1

TRAB_2.1

TRAB_2_1

Nif Funcionario
Daniel

Executar

Resultado

ID_Funcionar: 7
Nome Funcionario: Daniel
Nome Departamento: Departamento 3

Figura 4:Resultado Apex_2_1

TRAB_2_2

Código PLSQL

```
create or replace PROCEDURE TRAB_2_2(NIF_F in FORNECEDOR.NIF_FORNECEDOR%type ) AS

    Nome_FCR funcionario.NOME%type;
    LOC DEPARTAMENTO.NOME_DEP%type;
    invalid_nif_f EXCEPTION;

BEGIN
    if NIF_F<=0 then
        RAISE invalid_nif_f;
    else
        SELECT NOME_FORNECEDOR, LOCALIDADE
        into Nome_FCR, LOC
        FROM FORNECEDOR
        RIGHT join CODIGO_POSTAL on FORNECEDOR.CODIGO_POSTAL=CODIGO_POSTAL.CODIGO
        where FORNECEDOR.NIF_FORNECEDOR= NIF_F;

        http.p('NOME_FORNECEDOR: ' || Nome_FCR || '<br />');
        http.p('LOCALIDADE: ' || LOC || '<br />');
```

```
end if;
```

```
EXCEPTION
```

```
  WHEN invalid_nif_f THEN
```

```
    http.p('Nif tem que ser maior que 0');
```

```
  WHEN NO_DATA_FOUND THEN
```

```
    http.p ('Nif não existe!');
```

```
  WHEN others THEN
```

```
    http.p('Error!');
```

```
END TRAB_2_2;
```

Output

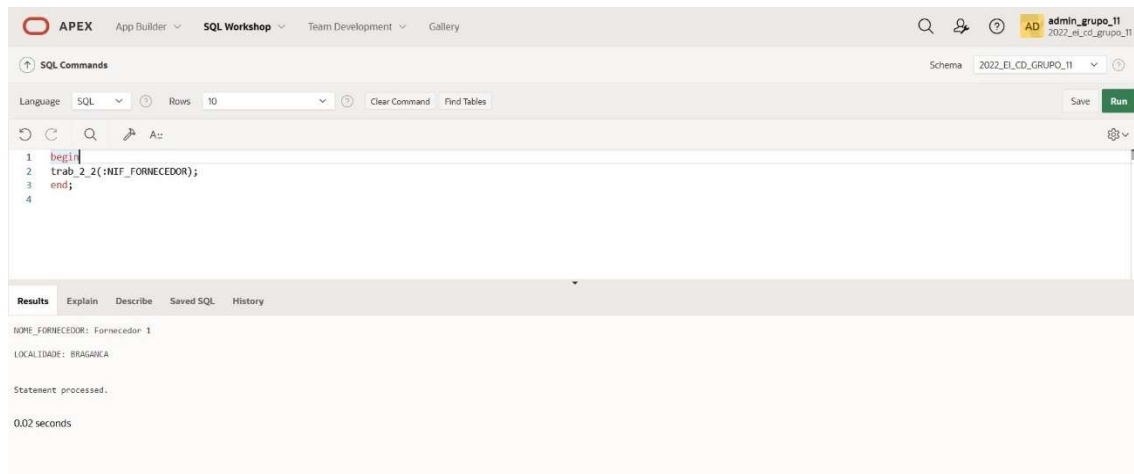


Figura 5: TRAB_2_2

Apex

TRAB_2.2

TRAB_2_2

Nif Fornecedor
Fornecedor 2

Executar

Resultado

NOME_FORNECEDOR: Fornecedor 2
LOCALIDADE: Santarem

Figura 6: Resultado Apex_2_2

TRAB_3_1

Código PLSQL

```
create or replace procedure TRAB_3_1(nif in Funcionario.NIF_FUNCIONARIO%type) as
reg_func Funcionario%rowtype;
begin
    select * into reg_func
    from Funcionario
    where salario = (select salario from Funcionario where NIF_FUNCIONARIO = nif);
    http.p('Funcionario: ' || reg_func.NOME || '<br />');
    http.p('Salario: ' || reg_func.salario || '<br />');

exception
    when NO_DATA_FOUND THEN
        HTP.P('Cargo inexistente');
    when OTHERS then
        http.p('!Erro Imprevisto!' || '<br />');
end TRAB_3_1;
```

Output

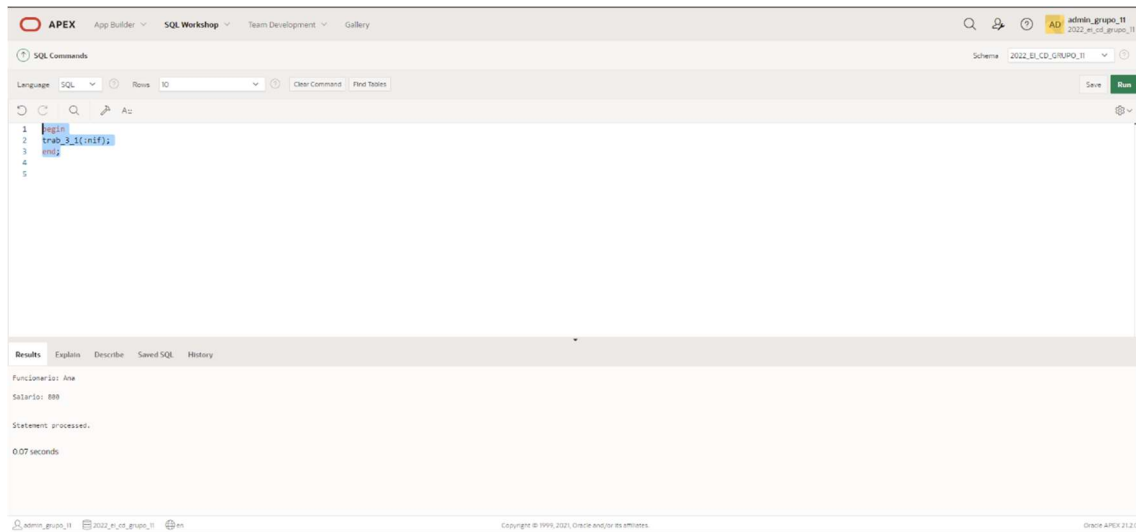


Figura 7:TRAb_3_1

APEX

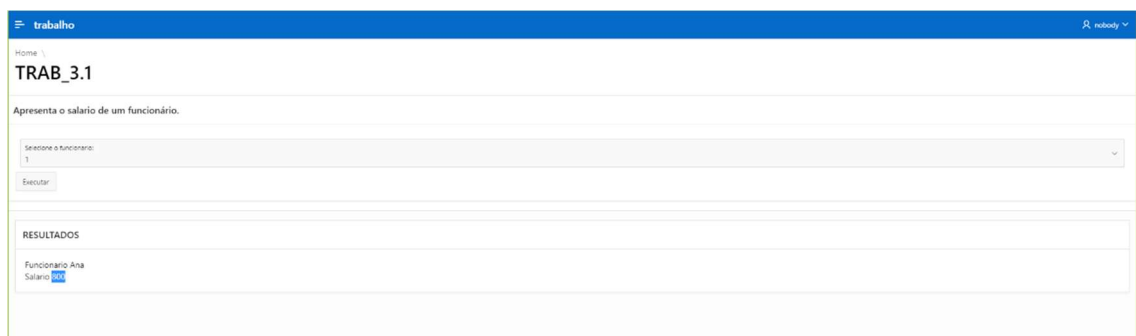


Figura 8: Resultado Apex_3_1

TRAB_3_2

Código PLSQL

```
create or replace procedure TRAB_3_2(prof Funcionario.cargo%type) as
reg_func Funcionario%rowtype;
profissao Funcionario.cargo%type;
begin
    profissao := prof;
    if prof is null then
        http.p('O Cargo não pode ser nulo' || '<br />');
    end if;
    select salario into reg_func.salario
    from Funcionario
    where salario in(
        select max(salario) as max_sal
        from Funcionario
```

```

where cargo = profissao);

    http.p('O salario mais alto dos funcionarios com a profissao: ' || to_char(profissao) || ' é: ' ||
to_char(reg_func.salario) || '<br />');
exception
    when OTHERS then
        http.p('!Erro Imprevisto!' || '<br />');
end TRAB_3_2;

```

Output

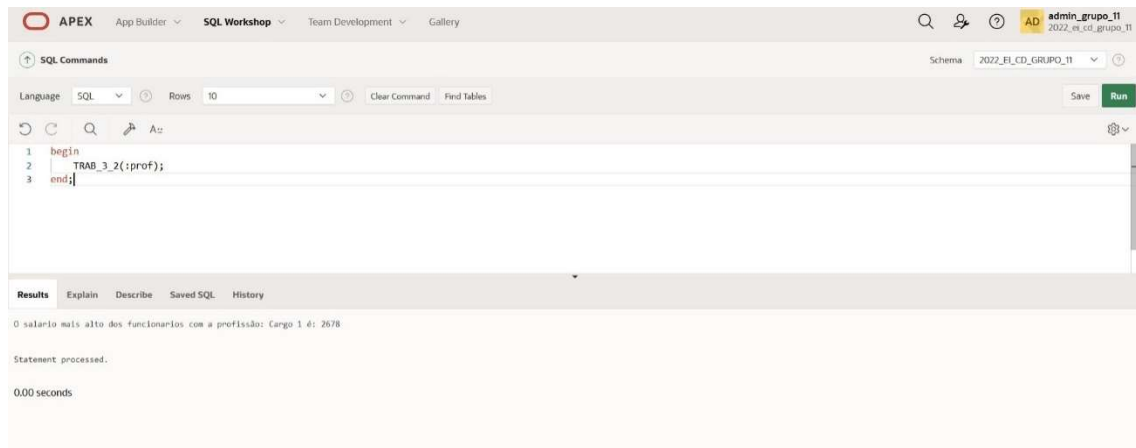


Figura 9:TRAb_3_2

APEX



Figura 10: Resultado Apex_3_2

TRAB_4_1

Código PLSQL

create or replace procedure TRAB_4_1(n_func in Funcionario.nif_funcionario%type, taxa_baixa in number, taxa_media in number, taxa_alta in number) as

```
    registo_func Funcionario%rowtype;
    sal Funcionario.salario%type;
    bonus number(8,2);
    total number(8,2);
    tempo number(3);
    nif_func Funcionario.nif_funcionario%type;
    SALARY_MISSING exception;
begin
    nif_func := n_func;

    select * into registo_func
    from Funcionario
    where nif_funcionario = nif_func;
    tempo := months_between(sysdate, registo_func.Data_Admicao);
    sal := registo_func.salario;
    if sal is null then
        raise SALARY_MISSING;
    end if;

    if (tempo < 48) then
        bonus := sal * taxa_baixa;
        total := sal + bonus;
    elsif (tempo < 96) then
        bonus := sal * taxa_media;
        total := sal + bonus;
    else
        bonus := sal * taxa_alta;
        total := sal + bonus;
    end if;

    http.p('Nome do Funcionário: ' || registo_func.nome || '<br />');
    http.p('Salário: ' || to_char(sal) || ' €' || '<br />');
    http.p('Tempo de Serviço: ' || to_char(tempo) || ' meses' || '<br />');
    http.p('Bónus: ' || to_char(bonus) || ' €' || '<br />');
    http.p('Salário + Bónus: ' || to_char(total) || ' €' || '<br />');
```

```

exception
when NO_DATA_FOUND then
    http.p('Funcionário: ' || to_char(n_func) || ' não existe.' || '<br />');
when SALARY_MISSING then
    http.p('O Funcionário ' || to_char(n_func) || ': não tem salário definido. Por favor, corrija
esta situação.' || '<br />');
when OTHERS then
    http.p('!Erro Imprevisto!' || '<br />');
end TRAB_4_1;

```

Output

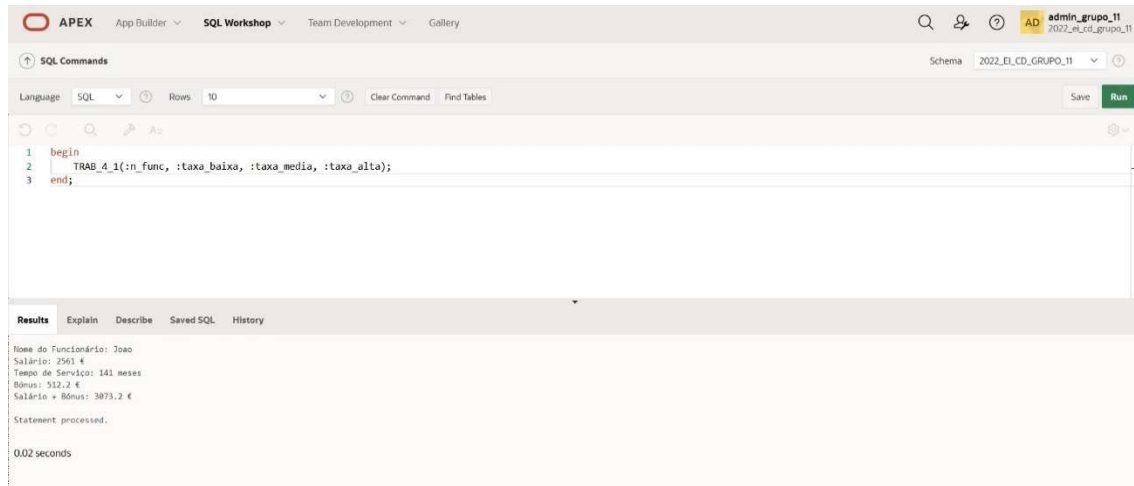


Figura 11: TRAB_4_1

APEX

TRAB_4.1

Calcular o bonus de um funcionario com base no tempo de serviço.

Selecione um Funcionário: Mario
Indique a taxa baixa: 0.05
Indique a taxa média: 0.10
Indique a taxa alta: 0.20
Executar

RESULTADOS

Nome do Funcionário: Mario
Salário: 2678 €
Tempo de Serviço: 43 meses
Bónus: 133.9 €
Salário + Bónus: 2811.9 €

Figura 12: Resultado Apex_4_1

TRAB_4_2

Código PLSQL

```
create or replace procedure TRAB_4_2(n_fun in Funcionario.nif_funcionario%type) as
```

```
    registo_func Funcionario%rowtype;  
    sal Funcionario.salario%type;  
    aumento number(8,2);  
    total number(8,2);  
    nif_func Funcionario.nif_funcionario%type;  
    SALARY_MISSING exception;
```



```

begin
    nif_func := n_fun;

    select * into registo_func
    from Funcionario
    where nif_funcionario = nif_func;

    sal := registo_func.salario;

    if sal is null then
        raise SALARY_MISSING;
    end if;

    case (registo_func.cargo)
        when 'Cargo 1' then
            if sal < 2000 then
                aumento := sal * 0.08;
                total := sal + aumento;
            else
                aumento := 0;
            end if;

        when 'Cargo 2' then
            if sal < 5500 then
                aumento := sal * 0.05;
                total := sal + aumento;
            else
                aumento := 0;
            end if;

        when 'Cargo 3' then
            if sal < 1500 then
                aumento := sal * 0.12;
                total := sal + aumento;
            else
                aumento := 0;
            end if;

    end case;

    if aumento = 0 then
        http.p('Este funcionario não tem aumento.' || '<br />');
    else
        http.p('Nome do Funcionário: ' || registo_func.nome || '<br />');
    end if;
end;

```

```

    http.p('Profissão: ' || registo_func.cargo || '<br />');
    http.p('Salário: ' || to_char(sal) || ' €' || '<br />');
    http.p('Aumento: ' || to_char(aumento) || ' €' || '<br />');
    http.p('Salário Após o Aumento: ' || to_char(total) || ' €' || '<br />');

end if;

exception
    when NO_DATA_FOUND then
        http.p('Funcionário: ' || to_char(n_fun) || ' não existe.' || '<br />');
    when SALARY_MISSING then
        http.p('O Funcionário ' || to_char(n_fun) || ': não tem salário definido. Por favor, corrija esta situação.' || '<br />');
    when OTHERS then
        http.p('!Erro Imprevisto!' || '<br />');
end TRAB_4_2;

```

Output

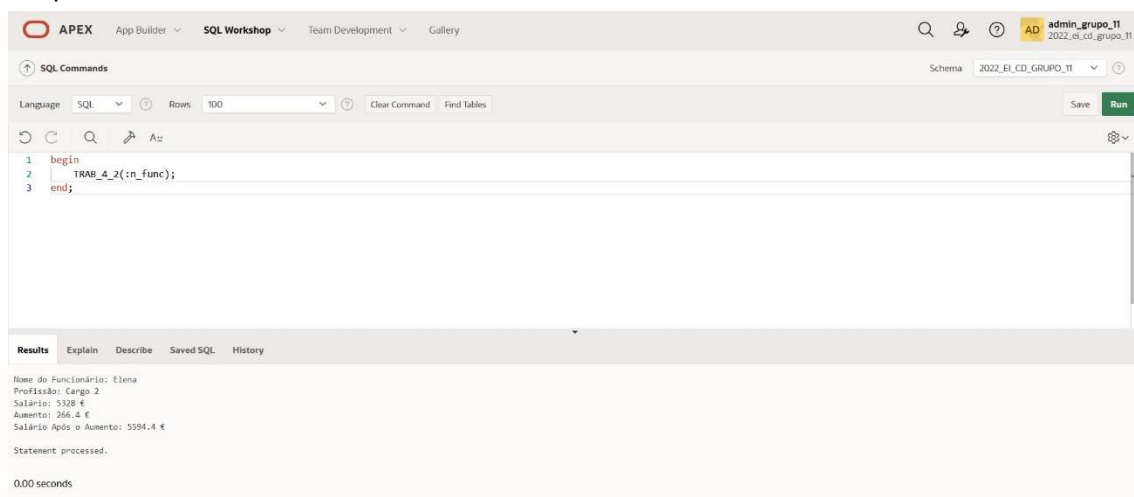


Figura 13: TRAB_4_2

APEX

TRAB_4.2

Calcular o aumento de um funcionário em função da sua profissão e salario atual.

Selecione um Funcionário:

Andreia

Executar

RESULTADOS

Nome do Funcionário: Andreia

Profissão: Cargo 3

Salário: 1239 €

Aumento: 148,68 €

Salário Após o Aumento: 1387.68 €

Figura 14:Resultado Apex_4_2

TRAB_5

Código PLSQL

```
create or replace procedure TRAB_5 as
  nif_cl Cliente.Nif%type;
  nome_cl Cliente.Nome%type;
  registo_tel Cliente_telefone%rowtype;
  registo_func Funcionario%rowtype;

  contador number;

  cursor clientes is
  select Cliente.Nif, Cliente.Nome, Cliente_telefone.telefone
  from Cliente, Cliente_telefone
  where Cliente.Nif = Cliente_telefone.Nif;
```

```

cursor funcionarios is
select nif_funcionario, nome, salario, id_dept
from Funcionario
order by salario desc;

begin
  http.p ('----- CLIENTES -----' || '<br />');
  open clientes;
  loop
    fetch clientes into nif_cl, nome_cl, registo_tel.telefone;
    exit when clientes%notfound;

    http.p('Cliente: ' || nif_cl || '<br />');
    http.p('Nome: ' || nome_cl || '<br />');
    http.p('Telefone: ' || registo_tel.telefone || '<br />');
    http.p(' ' || '<br />');

  end loop;
  close clientes;

  http.p ('----- FUNCIONARIOS -----' || '<br />');
  open funcionarios;
  for contador in 1..3 loop
    fetch funcionarios into registo_func.nif_funcionario, registo_func.nome,
    registo_func.salario, registo_func.id_dept;
    exit when funcionarios%notfound;

    http.p('Funcionario: ' || registo_func.nif_funcionario || '<br />');
    http.p('Nome: ' || registo_func.nome || '<br />');
    http.p('Salario: ' || registo_func.salario || '<br />');
    http.p('Departamento: ' || registo_func.id_dept || '<br />');
    http.p(' ' || '<br />');

  end loop;
  close funcionarios;

end TRAB_5;

```

Output

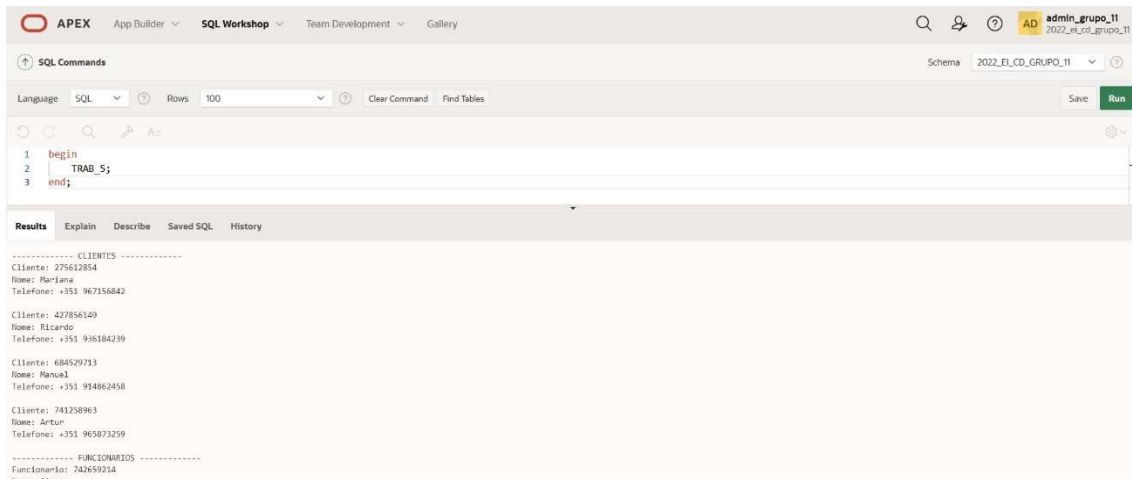


Figura 15: TRAB_5_1

APEX

trabalho

Home \

TRAB_5

Apresenta uma lista de todos os clientes e os 3 funcionários com maior salário.

RESULTADOS

----- CLIENTES -----

Cliente: 741258963
Nome: Artur
Telefone: +351 965873259

Cliente: 684529713
Nome: Manuel
Telefone: +351 914862458

Cliente: 427856149
Nome: Ricardo
Telefone: +351 936184239

Cliente: 275612854
Nome: Mariana
Telefone: +351 967156842

----- FUNCIONARIOS -----

Funcionario: 742659234

TRAB_6_1

Código PLSQL

```
create or replace procedure TRAB_6_1(n_f in Funcionario.nif_funcionario%type) as
```

```

registro_func Funcionario%rowtype;
nif_func Funcionario.nif_funcionario%type;
t_servico number(3);

begin
    nif_func := n_f;

    select * into registro_func
    from Funcionario
    where nif_funcionario = nif_func;

    t_servico := months_between(sysdate, registro_func.Data_Admicao);

    http.p('ID do Funcionário: ' || registro_func.id_funcionario || '<br />');
    http.p('Nome do Funcionário: ' || registro_func.nome || '<br />');
    http.p('Tempo de Serviço: ' || to_char(t_servico) || ' meses' || '<br />');

exception
    when NO_DATA_FOUND then
        http.p('Funcionário: ' || to_char(n_f) || ' não existe.' || '<br />');
    when OTHERS then
        http.p('!Erro Imprevisto!' || '<br />');
end TRAB_6_1;

```

Output

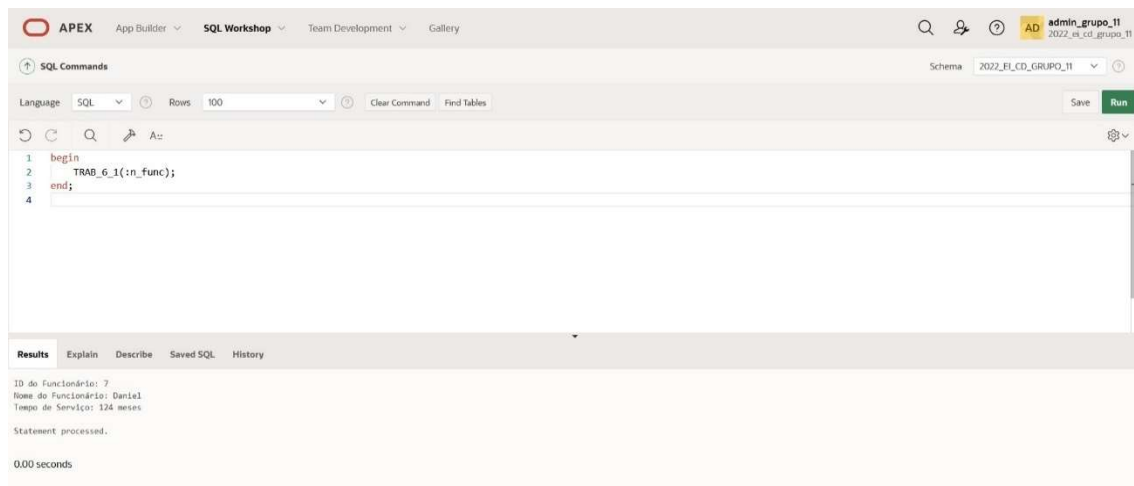


Figura 16: TRAB_6_1

APEX

TRAB_6

Apresenta o tempo (meses) de serviço de um funcionário.

Selecione um Funcionário:

Mario

Executar

RESULTADOS

ID do Funcionário: 5

Nome do Funcionário: Mario

Tempo de Serviço: 43 meses

Figura 17: Resultado Apex_6_1

TRAB_7_1

Código PLSQL

```
create or replace procedure TRAB_7_1 (dept_num in Departamento.id_dept%type) as
  nif Funcionario.nif_funcionario%type;
  nome Funcionario.nome%type;
  prof Funcionario.cargo%type;
  no_dept Departamento.Nome_dep%type;

  cursor cursor_dept is
    select Funcionario.nif_funcionario, Funcionario.nome, Funcionario.cargo,
    Departamento.Nome_dep
    from Funcionario, Departamento
    where Funcionario.id_dept = Departamento.id_dept
    and Departamento.id_dept = (dept_num);

begin
```

```

if dept_num is null then
    http.p('O Departamento não pode ser nulo' || '<br />');
end if;

open cursor_dept;
loop
    fetch cursor_dept into nif, nome, prof, no_dept;
    exit when cursor_dept%notfound;
    http.p('Funcionario: ' || nif || '<br />');
    http.p('Nome: ' || nome || '<br />');
    http.p('Departamento: ' || no_dept || '<br />');
    http.p('Profissão: ' || prof || '<br />');
    http.p(' ' || '<br />');

end loop;
close cursor_dept;

exception
    when NO_DATA_FOUND then
        http.p('O Departamento: ' || to_char(dept_num) || ' não existe.' || '<br />');
    when OTHERS then
        http.p('!Erro Imprevisto!' || '<br />');

end TRAB_7_1;

```

Output

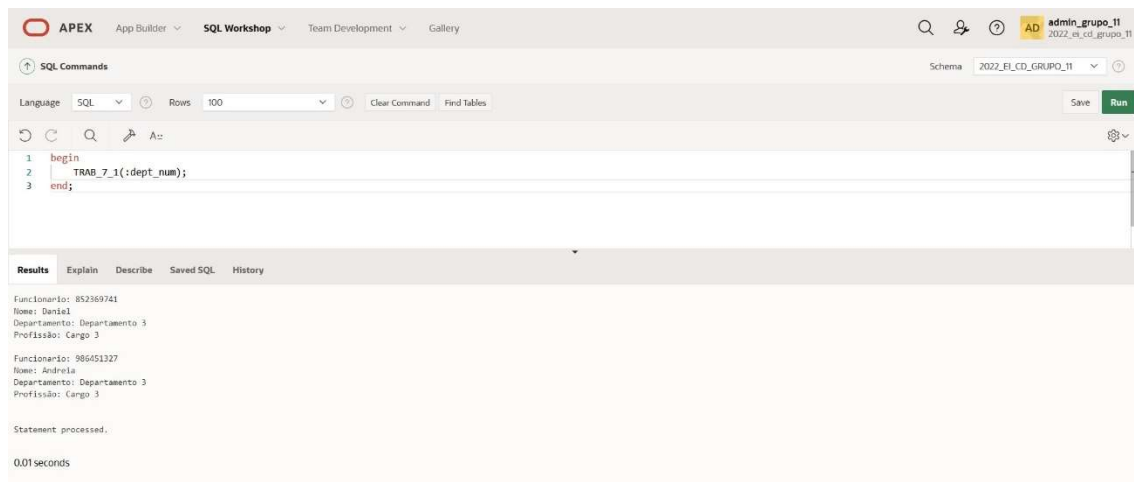


Figura 18: TRAB_7_1

APEX

TRAB_7.1

Apresenta a lista de funcionários de um departamento.

Selecione um Departamento:

Departamento 3

Executar

RESULTADOS

Funcionario: 852369741
Nome: Daniel
Departamento: Departamento 3
Profissão: Cargo 3

Funcionario: 986451327
Nome: Andreia
Departamento: Departamento 3
Profissão: Cargo 3

Figura 19: Resultado Apex_7_1

TRAB_7_2

Código PLSQL

```
create or replace procedure TRAB_7_2 (n_servico in Requisita.id_servico%type) as
  n_serv Requisita.id_servico%type;
  registo_cl_1 Cliente%rowtype;
  registo_cl_2 Cliente%rowtype;
  registo_serv_1 Requisita%rowtype;
  registo_serv_2 Requisita%rowtype;

  n_linhas number(2);
  cont number;
```

```

cursor cursor_1 is
select Cliente.Nif, Cliente.Nome, Requisita.id_servico
from Cliente, Requisita
where Cliente.Nif = Requisita.Nif
and Requisita.id_servico = n_serv;

cursor cursor_2 is
select Cliente.Nif, Cliente.Nome, Requisita.id_servico
from Cliente, Requisita
where Cliente.Nif = Requisita.Nif
and Requisita.id_servico = 3;

begin
n_serv := n_servico;
htp.p ('----- CURSOR 1 -----' || '<br />');

if n_servico is null then
    htp.p('O Serviço não pode ser nulo' || '<br />');
end if;

open cursor_1;
loop
    fetch cursor_1 into registo_cl_1.nif, registo_cl_1.nome, registo_serv_1.id_servico;
    exit when cursor_1%notfound;
    htp.p('Cliente: ' || registo_cl_1.nif || '<br />');
    htp.p('Nome: ' || registo_cl_1.nome || '<br />');
    htp.p('Servico: ' || registo_serv_1.id_servico || '<br />');
    htp.p(' ' || '<br />');

end loop;
close cursor_1;

htp.p ('----- CURSOR 2 -----' || '<br />');
open cursor_2;
for cont in 1..2 loop
    fetch cursor_2 into registo_cl_2.nif, registo_cl_2.nome, registo_serv_2.id_servico;
    exit when cursor_2%notfound;
    htp.p('Cliente: ' || registo_cl_2.nif || '<br />');
    htp.p('Nome: ' || registo_cl_2.nome || '<br />');
    htp.p('Servico: ' || registo_serv_2.id_servico || '<br />');
    htp.p(' ' || '<br />');

end loop;
n_linhas := cursor_2%rowcount;

```

```

http.p('Numero de Clientes: ' || n_linhas || '<br />');
close cursor_2;

exception
  when NO_DATA_FOUND then
    http.p('O Servico: ' || to_char(n_servico) || ' não existe.' || '<br />');
  when OTHERS then
    http.p('!Erro Imprevisto!' || '<br />');

end TRAB_7_2;

```

Output

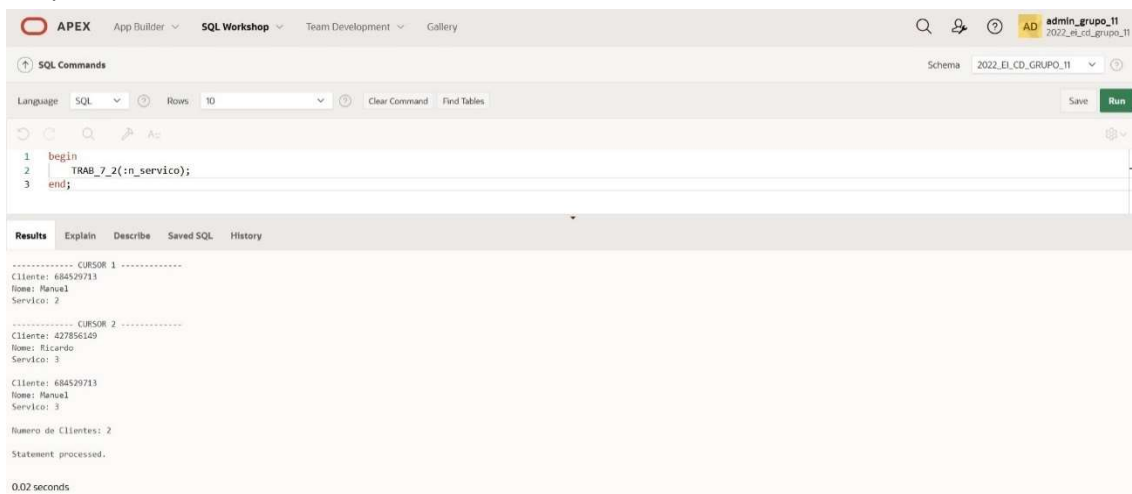


Figura 20: TRAB_7_2

APEX

TRAB_7.2

Apresenta a lista de clientes que requisitaram um serviço.

Selecione um Serviço:

Servico 1

Executar

Cliente: 275612854

Nome: Mariana

Servico: 1

Cliente: 741258963

Nome: Artur

Servico: 1

----- CURSOR 2 -----

Cliente: 427856149

Nome: Ricardo

Servico: 3

Cliente: 684529713

Nome: Manuel

Servico: 3

Numero de Clientes: 2

Figura 21: Resultado Apex_7_2

TRAB_7_3

Código PLSQL

```
create or replace procedure TRAB_7_3 (n_dep in Funcionario.id_dept%type) as
  n_depart Funcionario.id_dept%type;
  registo_func1 Funcionario%rowtype;
  registo_func2 Funcionario%rowtype;
  NO_DEPT_FOUND exception;

  cursor cursor_max is
```

```

select nif_funcionario, nome, salario, id_dept
from Funcionario
where salario in (
    select max(salario) as max_sal
    from Funcionario
    where id_dept = n_depart);

cursor cursor_min is
select nif_funcionario, nome, cargo, salario, id_dept
from Funcionario
where salario in (
    select max(salario) as min_sal
    from Funcionario
    where id_dept = 1);

begin
    n_depart := n_dep;
    http.p('----- CURSOR 1 -----' || '<br />');

    if n_dep is null then
        http.p('O Departamento não pode ser nulo' || '<br />');
    end if;
    open cursor_max;
    loop
        fetch cursor_max into registo_func1.nif_funcionario, registo_func1.nome,
        registo_func1.salario, registo_func1.id_dept;
        exit when cursor_max%notfound;
        http.p('Funcionário: ' || to_char(registo_func1.nif_funcionario) || '<br />');
        http.p('Nome: ' || to_char(registo_func1.nome) || '<br />');
        http.p('Salário: ' || to_char(registo_func1.salario) || '€' || '<br />');
        http.p('Departamento: ' || to_char(registo_func1.id_dept) || '<br />');
        http.p(' ' || '<br />');

    end loop;
    close cursor_max;

    http.p('----- CURSOR 2 -----' || '<br />');
    open cursor_min;
    loop
        fetch cursor_min into registo_func2.nif_funcionario, registo_func2.nome,
        registo_func2.cargo, registo_func2.salario, registo_func2.id_dept;
        exit when cursor_min%notfound;
        http.p('Funcionário: ' || to_char(registo_func2.nif_funcionario) || '<br />');
        http.p('Nome: ' || to_char(registo_func2.nome) || '<br />');
        http.p('Profissão: ' || to_char(registo_func2.cargo) || '<br />');
        http.p('Salário: ' || to_char(registo_func2.salario) || '€' || '<br />');

```

```

    http.p('Departamento: ' || to_char(registo_func2.id_dept) || '<br />');
    http.p(' ');

end loop;
close cursor_min;

exception
    when NO_DATA_FOUND then
        http.p('O Departamento: ' || to_char(n_dep) || ' não existe' || '<br />');
    when OTHERS then
        http.p('!Erro Imprevisto!' || '<br />');

end TRAB_7_3;

```

Output

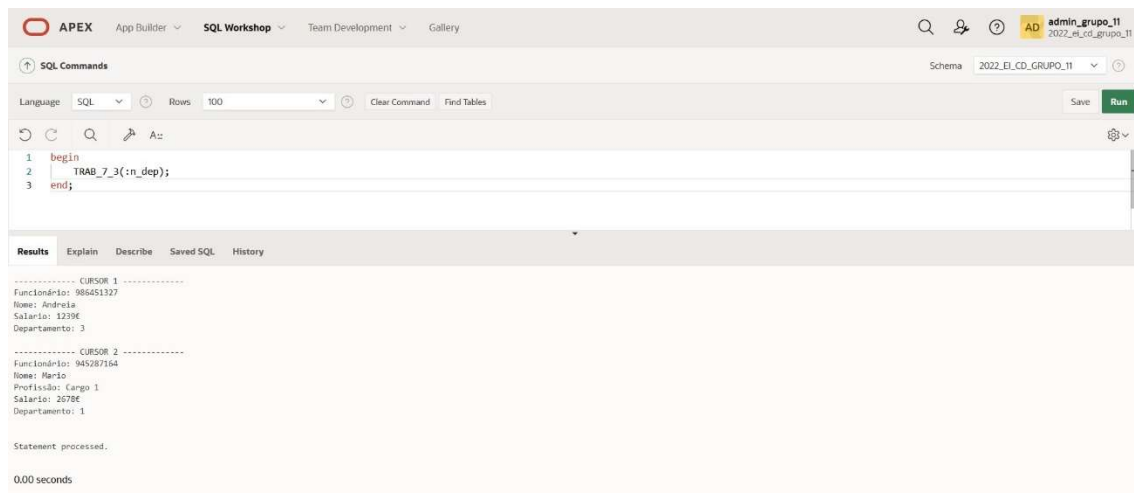


Figura 22: TRAB_7_3

APEX

TRAB_7.3

Apresenta o funcionário com o maior salario de um departamento.

Selecione um Departamento:

Departamento 3

Executar

RESULTADOS

----- CURSOR 1 -----

Funcionário: 986451327

Nome: Andreia

Salario: 1239€

Departamento: 3

----- CURSOR 2 -----

Funcionário: 945287164

Nome: Mario

Profissão: Cargo 1

Salario: 2678€

Departamento: 1

Figura 23: Resultado Apex_7_3

TRAB_7_4

Código PLSQL

```
create or replace procedure TRAB_7_4(n_fornecedor in Fornecedor.nif_fornecedor%type) as
  type forne_refcur_typ is ref cursor return Fornecedor%rowtype;
  fornecedor_cursor forne_refcur_typ;
  procedure fornecedor_cur(fornecedor_cursor forne_refcur_typ) is
    forne Fornecedor%rowtype;
  begin
    loop
      fetch fornecedor_cursor into forne;
```

```

        exit when fornecedor_cursor%notfound;
        http.p('Fornecedor: ' || forne.nome_fornecedor || '<br />');
        http.p('Codigo Postal: ' || forne.codigo_postal || '<br />');
    end loop;
end;
begin
    open fornecedor_cursor for
    select *
    from Fornecedor
    where nif_fornecedor = n_fornecedor;
    fornecedor_cur(fornecedor_cursor);
    close fornecedor_cursor;
end TRAB_7_4;

```

Output

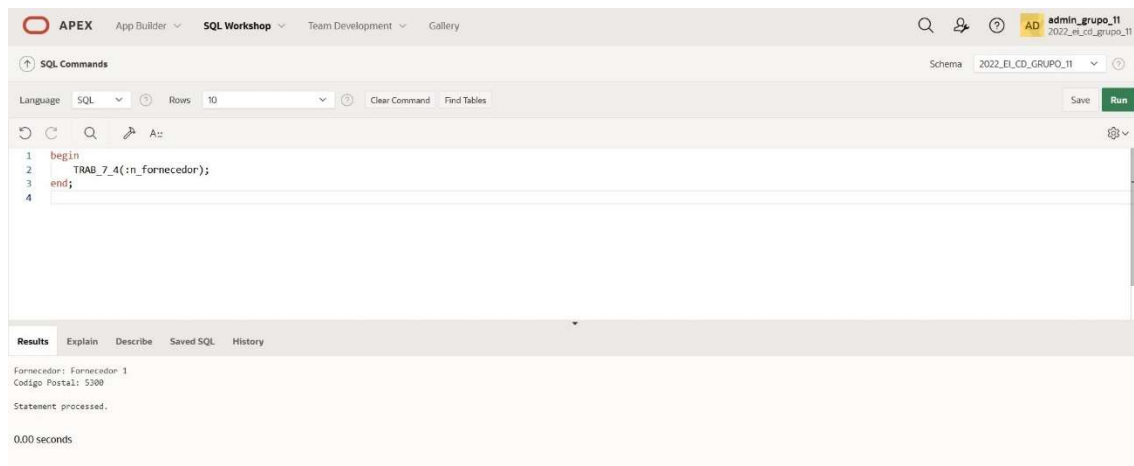


Figura 24: TRAB_7_4

APEX

TRAB_7.4

Apresenta um fornecedor e o respetivo código postal.

Selecione um Fornecedor:

Fornecedor 2

Executar

RESULTADOS

Fornecedor: Fornecedor 2
Codigo Postal: 2140

Figura 25: Resultado Apex_7_4

TRAB_8_1

Código PLSQL

```
create or replace procedure TRAB_8_1 (nome_cl in Cliente.Nome%type, nif_cl in
Cliente.Nif%type, tel in Cliente_telefone.telefone%type, mail in Cliente_Email.Email%type) as

begin
    if nome_cl is null then
        http.p('O nome do cliente não pode ser nulo.' || '<br />');
    elsif nif_cl is null then
        http.p('O nif do cliente não pode ser nulo.' || '<br />');
    elsif tel is null then
        http.p('O telefone do cliente não pode ser nulo.' || '<br />');
    elsif mail is null then
        http.p('O email do cliente não pode ser nulo.' || '<br />');
    else
        insert into Cliente(Nome, Nif)
```

```

values (nome_cl, nif_cl);

insert into cliente_telefone(telefone, Nif)
values (tel, nif_cl);

insert into cliente_Email(Email, Nif)
values (mail, nif_cl);

http.p('O registo de cliente foi inserido com sucesso.' || '<br />');
end if;

exception
when DUP_VAL_ON_INDEX then
    http.p('Já existe um cliente com o nif ' || to_char(nif_cl) || '. Por favor, indique um
número que ainda não esteja a ser usado.' || '<br />');
when others then
    http.p('Erro imprevisto.' || '<br />');

end TRAB_8_1;

```

Output

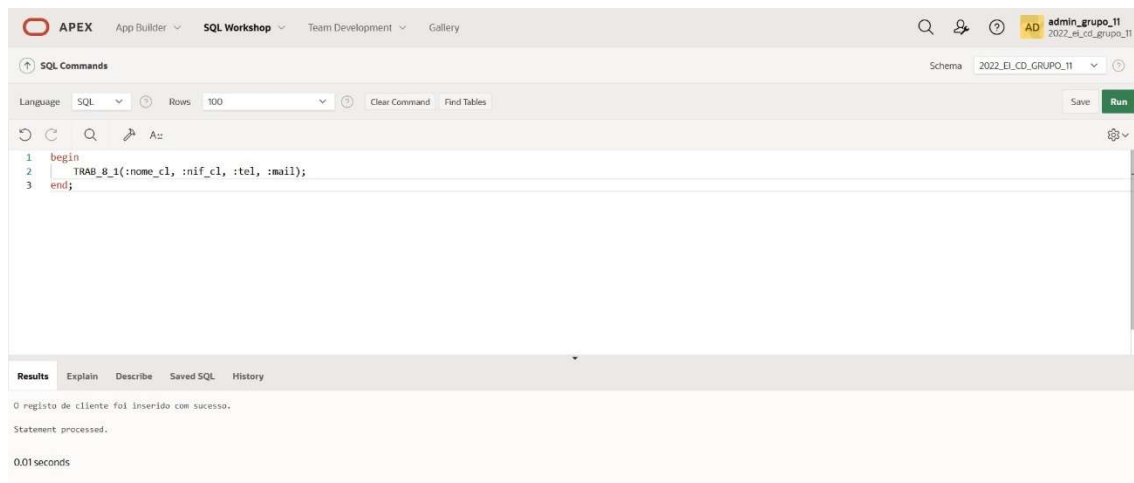


Figura 26: TRAB_8_1

APEX

TRAB_8.1

Inserir novo Cliente.

Insira o nome do cliente:

Teste

Insira o nif do cliente:

999999999

Insira o telefone do cliente:

969999999

Insira o email do cliente:

teste

Submeter Novo Cliente

RESULTADOS

O registo de cliente foi inserido com sucesso.

Figura 27: Resultado Apex_8_1

TRAB_8_2

Código PLSQL

```
create or replace procedure TRAB_8_2 (nif_client in Cliente.Nif%type) as
    reg_cliente Cliente%rowtype;
    reg_cl_telefone cliente_telefone%rowtype;
    reg_cl_email cliente_Email%rowtype;

    CHILD_RECORD_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(CHILD_RECORD_FOUND, -02292);
```

```

begin

if nif_client is null then
    http.p('O nif do cliente não pode ser nulo.' || '<br />');
else

    select *
    into reg_cl_telefone
    from cliente_telefone
    where Nif = nif_client;

    delete from cliente_telefone
    where Nif = nif_client;

    select *
    into reg_cl_email
    from cliente_Email
    where Nif = nif_client;

    delete from cliente_Email
    where Nif = nif_client;

    select *
    into reg_cliente
    from Cliente
    where Nif = nif_client;

    delete from Cliente
    where Nif = nif_client;

    http.p('O Cliente ' || to_char(nif_client) || ' e os seus respetivos contactos foram
removidos com sucesso.' || '<br />');
end if;

exception
    when NO_DATA_FOUND then
        http.p('O Cliente ' || to_char(nif_client) || ' não existe.' || '<br />');
    when CHILD_RECORD_FOUND then
        http.p('O Cliente ' || to_char(nif_client) || ' não pode ser removido, pois está associado a
outra tabela' || '<br />');
    when others then
        http.p('Erro imprevisto.' || '<br />');

```

```
end TRAB_8_2;
```

Output

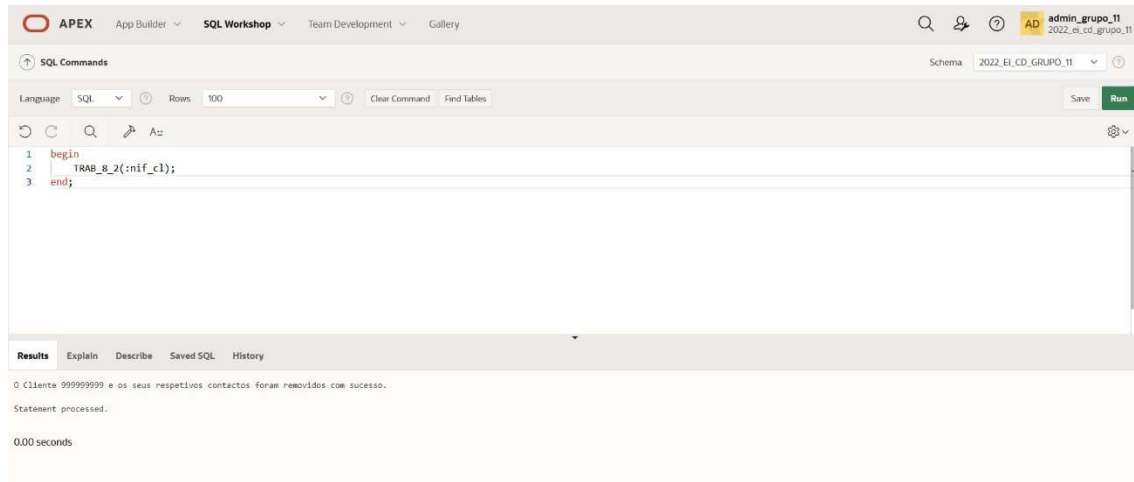


Figura 28:TRAb_8_2

APEX



TRAB_8.2

Apagar um Cliente.



Figura 29: Resultado Apex_8_2

TRAB_8_3

Código PLSQL

```
create or replace procedure TRAB_8_3 (n_client in Cliente.Nif%type, nome_client in
Cliente.Nome%type, tel_client in Cliente_telefone.telefone%type, mail_client in
Cliente_Email.Email%type) as
```

```
    reg_cliente Cliente%rowtype;
```

```
    reg_cl_telefone Cliente_telefone%rowtype;
```

```
    reg_cl_email Cliente_Email%rowtype;
```

```
    CHILD_RECORD_FOUND EXCEPTION;
```

```
    PRAGMA EXCEPTION_INIT(CHILD_RECORD_FOUND, -02292);
```

```
begin
```

```
    if n_client is null then
```

```
        http.p('O nif do cliente não pode ser nulo.' || '<br />');
```

```
    elsif nome_client is null then
```

```
        http.p('O nome do cliente não pode ser nulo.' || '<br />');
```

```
    elsif tel_client is null then
```

```
        http.p('O telefone do cliente não pode ser nulo.' || '<br />');
```

```
    elsif mail_client is null then
```

```
        http.p('O email do cliente não pode ser nulo.' || '<br />');
```

```
    else
```

```
        SELECT *
```

```
        INTO reg_cliente
```

```
        FROM Cliente
```

```
        WHERE Nif = n_client;
```

```
        update Cliente
```

```
        set Cliente.Nome = nome_client
```

```
        where Cliente.Nif = n_client;
```

```
        SELECT *
```

```
        INTO reg_cl_telefone
```

```
        FROM Cliente_telefone
```

```
        WHERE Nif = n_client;
```

```
        update Cliente_telefone
```

```
        set Cliente_telefone.telefone = tel_client
```

```
        where Cliente_telefone.Nif = n_client;
```

```

SELECT *
INTO reg_cl_email
FROM Cliente_Email
WHERE Nif = n_client;

update Cliente_Email
set Cliente_Email.Email = mail_client
where Cliente_Email.Nif = n_client;

http.p('Os dados do cliente ' || to_char(n_client) || ' foram alterados com sucesso.' || '<br
/>');
end if;

exception
when NO_DATA_FOUND then
    http.p('O Cliente ' || to_char(n_client) || ' não existe.' || '<br />');
when CHILD_RECORD_FOUND then
    http.p('O Cliente ' || to_char(n_client) || ' não pode ser removido, pois está associado a
outra tabela' || '<br />');
when others then
    http.p('Erro imprevisto.' || '<br />');

end TRAB_8_3;

```

Output

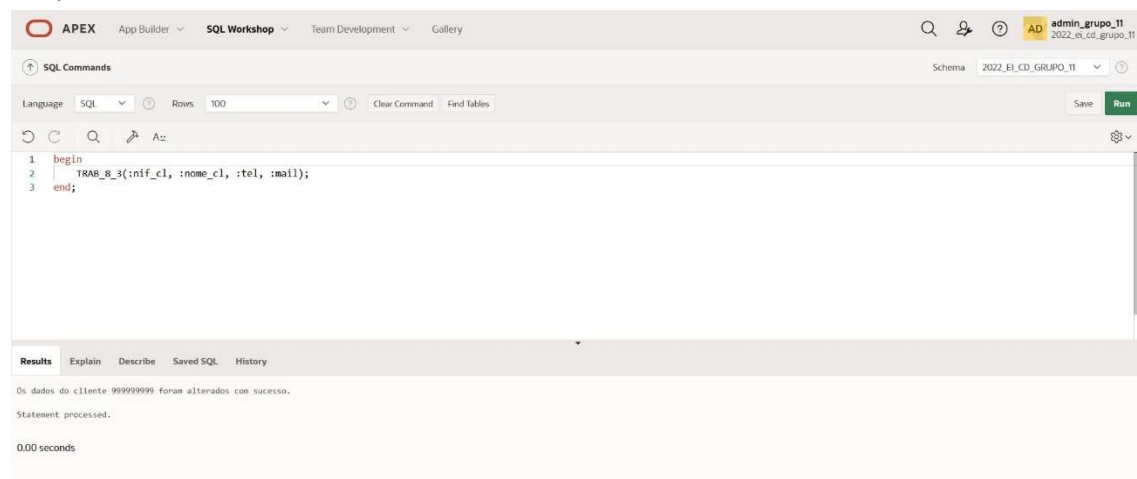


Figura 30:TRAB_8_3

APEX

TRAB_8.3

Alterar dados de um Cliente.

Selecione um cliente:	Teste
Insira o novo nome do cliente:	Teste_1
Insira o novo telefone do cliente:	961474582
Insira o novo email do cliente:	teste@gmail.com
Submeter Alterações	

RESULTADOS

Os dados do cliente 111111111 foram alterados com sucesso.

Figura 31:RESULTADO APEX_8_3

TRAB_9_1

Código PLSQL

```
create or replace procedure TRAB_9_1(id_func in Funcionario.id_funcionario%type, nome_func
Funcionario.nome%type, prof_func Funcionario.cargo%type, n_sec number, n_gabinete
number) as
type loc_rec is record(
  gabinete number(2),
  sec number(2)
);
type func_rec is record(
  id_f Funcionario.id_funcionario%type,
  nome_f Funcionario.nome%type,
  cargo_f Funcionario.cargo%type,
  loc loc_rec
```



```

);
func func_rec;
begin
    func.id_f := id_func;
    func.nome_f := nome_func;
    func.cargo_f := prof_func;
    func.loc.sec := n_sec;
    func.loc.gabinete := n_gabinete;
    if id_func is null then
        http.p('O id do funcionario não pode ser nulo.' || '<br />');

    elsif nome_func is null then
        http.p('O nome do funcionario não pode ser nulo.' || '<br />');

    elsif prof_func is null then
        http.p('O cargo do funcionario não pode ser nulo.' || '<br />');

    elsif n_sec is null then
        http.p('O numero da secção do funcionario não pode ser nulo.' || '<br />');

    elsif n_gabinete is null then
        http.p('O numero do gabinete do funcionario não pode ser nulo.' || '<br />');
    else
        http.p('Funcionario: ' || func.id_f || '<br />');
        http.p('Nome: ' || func.nome_f || '<br />');
        http.p('Cargo: ' || func.cargo_f || '<br />');
        http.p('Secção: ' || func.loc.sec || '<br />');
        http.p('Gabinete: ' || func.loc.gabinete || '<br />');
    end if;
exception
    when others then
        http.p('Erro Imprevisto' || '<br />');
end TRAB_9_1;

```

Output

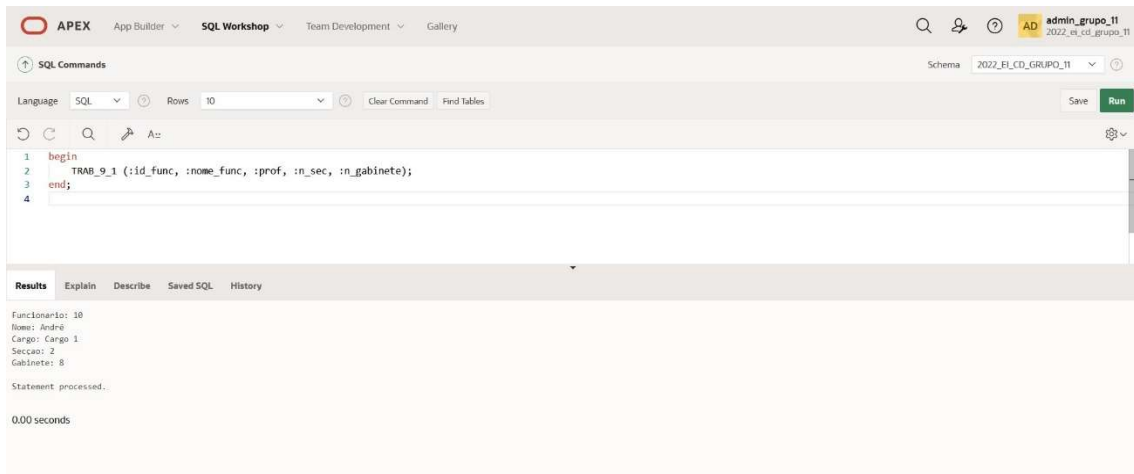


Figura 32:TRAB_9_1

APEX

TRAB_9.1

Record para dados do tipo Funcionario.

Insira o id do Funcionário:	99
Insira o nome do Funcionário:	Teste
Insira a profissão do Funcionário:	teste
Insira a secção do Funcionário:	99
Insira o gabinete do Funcionário:	99
<button>Executar</button>	

Funcionario: 99
Nome: Teste
Cargo: teste
Secção: 99
Gabinete: 99

 Home

Figura 33:RESULTADO APEX_9_1

TRAB_9_2

Código PLSQL

```
create or replace procedure TRAB_9_2 as
type cod_postal_rec is record(
  cod Codigo_Postal.codigo%type,
  loc Codigo_Postal.localidade%type
);
```

```

codigo_post cod_postal_rec;
cursor cur_postal is
select codigo, localidade
from Codigo_Postal;
begin
    open cur_postal;
    loop
        fetch cur_postal into codigo_post.cod, codigo_post.loc;
        exit when cur_postal%notfound;
        http.p('Codigo Postal: ' || codigo_post.cod || '<br />');
        http.p('Localidade: ' || codigo_post.loc || '<br />');
        http.p(' ' || '<br />');
    end loop;
exception
    when no_data_found then
        http.p('Não existem codigos postais disponiveis.' || '<br />');
    when others then
        http.p('Erro Imprevisto.' || '<br />');
end TRAB_9_2;

```

Output

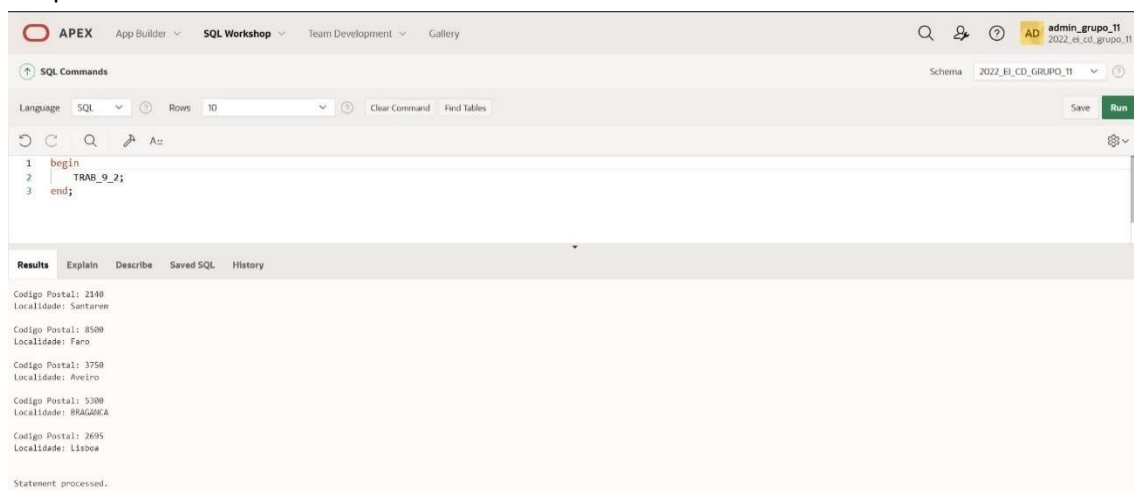


Figura 34:TRAB_9_2

TRAB_9.2

Record com cursor para dados do tipo Codigo_Postal.

RESULTADOS

Codigo Postal: 2140
Localidade: Santarem

Codigo Postal: 8500
Localidade: Faro

Codigo Postal: 3750
Localidade: Aveiro

Codigo Postal: 5300
Localidade: BRAGANCA

Codigo Postal: 2695
Localidade: Lisboa

Figura 35:RESULTADO APEX_9_2

TRAB_10_1

Código PLSQL

```
create or replace procedure TRAB_10_1 as
type dept_array is array(10) of varchar2(10);
dept dept_array;
reg_dept Departamento%rowtype;
total number;
begin
    dept := dept_array(1, 2, 3);
```

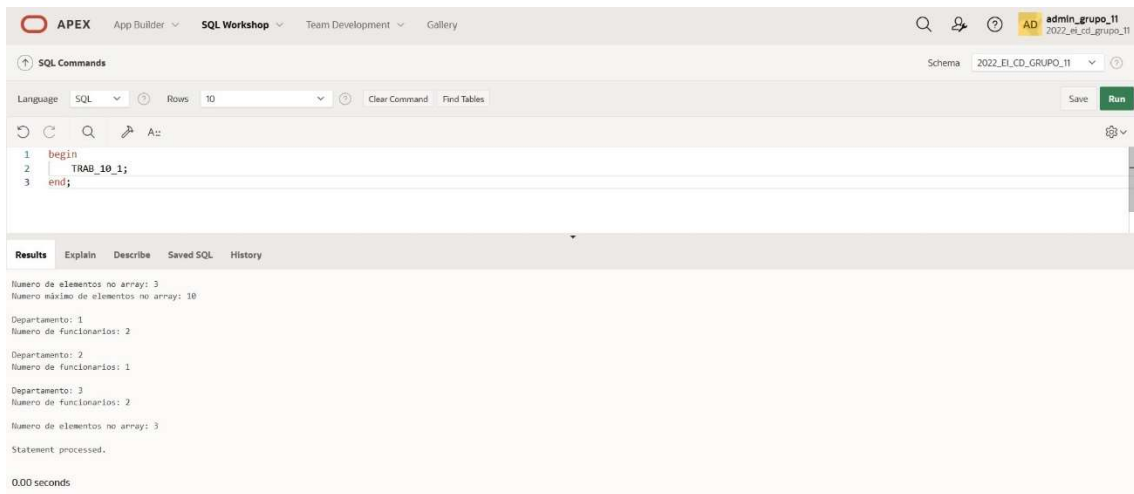
```

http.p('Numero de elementos no array: ' || dept.count || '<br />');
http.p('Numero máximo de elementos no array: ' || dept.limit || '<br />');
http.p(' ' || '<br />');
for i in dept.first..dept.last loop
    select count(*) into total
    from funcionario
    where id_dept = dept(i);
    http.p('Departamento: ' || rpad(dept(i), 10) || '<br />');
    http.p('Numero de funcionarios: ' || to_char(total) || '<br />');
    http.p(' ' || '<br />');
end loop;
http.p('Numero de elementos no array: ' || dept.count || '<br />');

exception
    when others then
        http.p('Erro Imprevisto!' || '<br />');
end TRAB_10_1;

```

Output



The screenshot shows the APEX SQL Workshop interface. The SQL Commands pane contains the following code:

```

1 begin
2   TRAB_10_1;
3 end;

```

The Results pane shows the output of the procedure:

```

Numero de elementos no array: 3
Numero máximo de elementos no array: 10

Departamento: 1
Numero de funcionarios: 2

Departamento: 2
Numero de funcionarios: 1

Departamento: 3
Numero de funcionarios: 2

Numero de elementos no array: 3
Statement processed.

0.00 seconds

```

Figura 36: TRAB_10_1

TRAB_10.1

Apresenta o numero de funcionários de cada departamento.

RESULTADOS

Numero de elementos no array: 3
Numero máximo de elementos no array: 10

Departamento: 1
Numero de funcionarios: 2

Departamento: 2
Numero de funcionarios: 1

Departamento: 3
Numero de funcionarios: 2

Numero de elementos no array: 3

Figura 37:RESULTADO APEX_10_1

TRAB_10_2

Código PLSQL

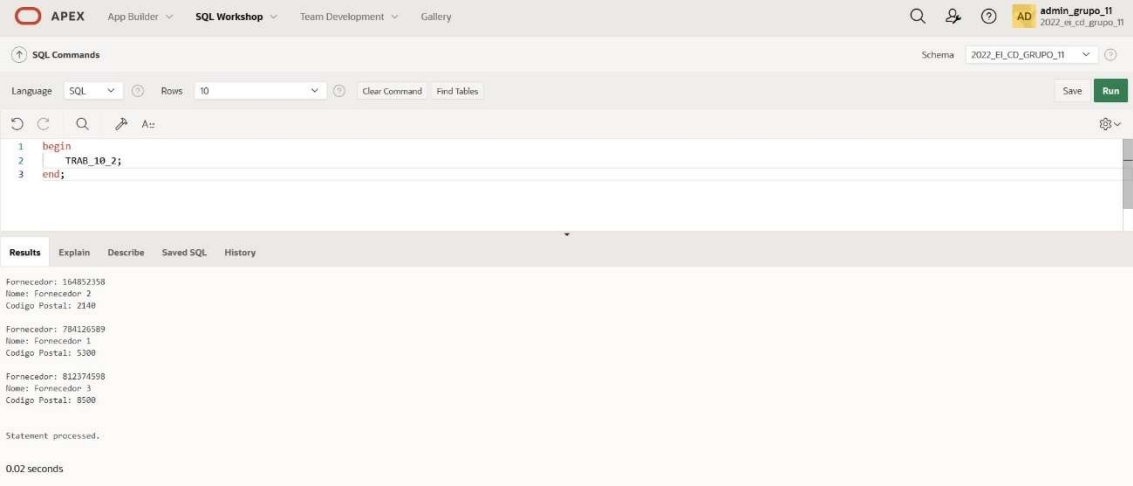
```
create or replace procedure TRAB_10_2 as
cursor cursor_for is
select *
  from Fornecedor;
rec_for cursor_for%rowtype;
type for_array is table of cursor_for%rowtype;
f_array for_array := for_array();
```

```

i number := 1;
begin
  open cursor_for;
  loop
    fetch cursor_for into rec_for;
    exit when cursor_for%notfound;
    f_array.extend(1);
    f_array(i) := rec_for;
    i := i + 1;
  end loop;
  close cursor_for;
  for i in f_array.first..f_array.last loop
    http.p('Fornecedor: ' || to_char(f_array(i).nif_fornecedor) || '<br />');
    http.p('Nome: ' || to_char(f_array(i).nome_fornecedor) || '<br />');
    http.p('Codigo Postal: ' || to_char(f_array(i).codigo_postal) || '<br />');
    http.p(' ' || '<br />');
  end loop;
exception
  when others then
    http.p('Erro Imprevisto!' || '<br />');
end TRAB_10_2;

```

Output



The screenshot shows the APEX SQL Workshop interface. The SQL Commands window contains the following code:

```

1 begin
2   TRAB_10_2;
3 end;

```

The Results window shows the output of the procedure, which prints the details of three suppliers (Fornecedores) in a formatted manner:

```

Fornecedor: 164852358
Nome: Fornecedor 2
Codigo Postal: 2148

Fornecedor: 784126589
Nome: Fornecedor 1
Codigo Postal: 5388

Fornecedor: 812374598
Nome: Fornecedor 3
Codigo Postal: 8500

Statement processed.

0.02 seconds

```

Figura 38: TRAB_10_2

TRAB_10.2

Apresenta a lista de Fornecedores.

RESULTADOS

Fornecedor: 164852358
Nome: Fornecedor 2
Codigo Postal: 2140

Fornecedor: 784126589
Nome: Fornecedor 1
Codigo Postal: 5300

Fornecedor: 812374598
Nome: Fornecedor 3
Codigo Postal: 8500

Figura 39_RESULTADO APEX_10_2

Funções

TRAB_6_2

Código PLSQL

```
create or replace FUNCTION TRAB_6_2 (cliente_nif Cliente.nif%TYPE) RETURN varchar2
IS
    nome_cliente varchar2(50);
BEGIN
    SELECT INITCAP (nome)
    INTO nome_cliente
    FROM cliente
    WHERE nif = cliente_nif;
```

```

        RETURN nome_cliente;
EXCEPTION
    WHEN OTHERS THEN
        RETURN NULL;
END TRAB_6_2;

```

Output

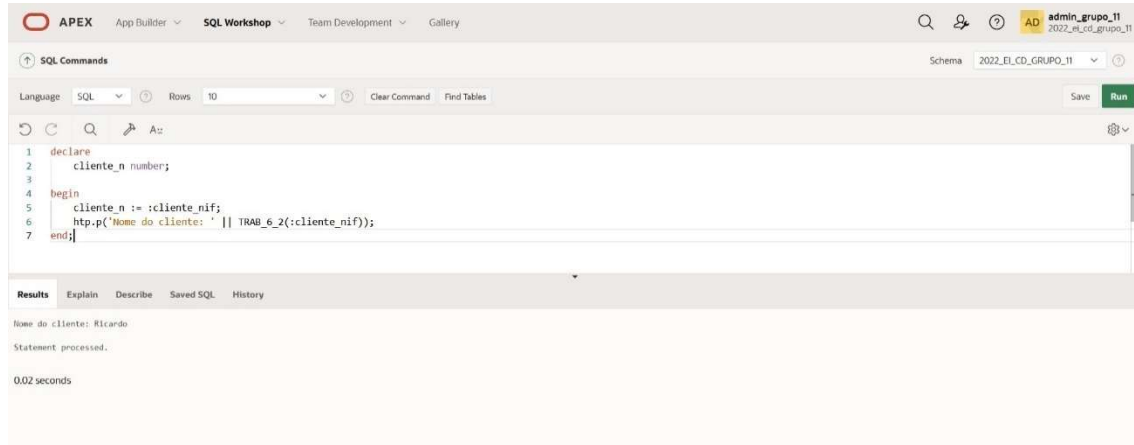


Figura 40: TRAB_6_2

APEX

trabalho

Home \

TRAB_6.2

Apresenta o nome de um cliente.

Selecione o nif do Cliente:

960097032

Executar

RESULTADOS

Nome do cliente: Guilherme

Figura 41: Resultado Apex_6_2

Triggers

Para criar os triggers pedidos no trabalho nos vimos na necessidade de adicionar mais algumas tabelas na base de dados sendo elas: Logs e Auditoria.

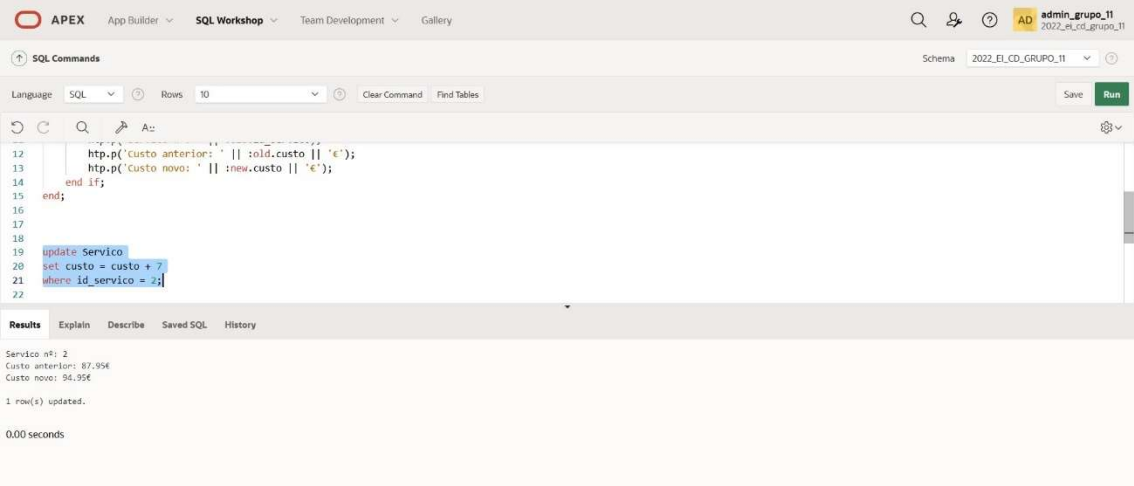
TRAB_11_1

Código PLSQL

```
create or replace trigger TRAB_11_1
before insert or update on Servico
for each row
when (new.id_servico > 0)
begin
    if to_char(sysdate, 'HH24') between '15' and '22' then
        RAISE_APPLICATION_ERROR(-20205, 'Alterações na tabela Servico não são permitidas
entre as 15h e as 22h.' || '<br />');

    elsif INSERTING then
        http.p('Registo inserido com sucesso!' || '<br />');
    else
        http.p('Servico nº: ' || :old.id_servico || '<br />');
        http.p('Custo anterior: ' || :old.custo || '€' || '<br />');
        http.p('Custo novo: ' || :new.custo || '€' || '<br />');
    end if;
end TRAB_11_1;
```

Output



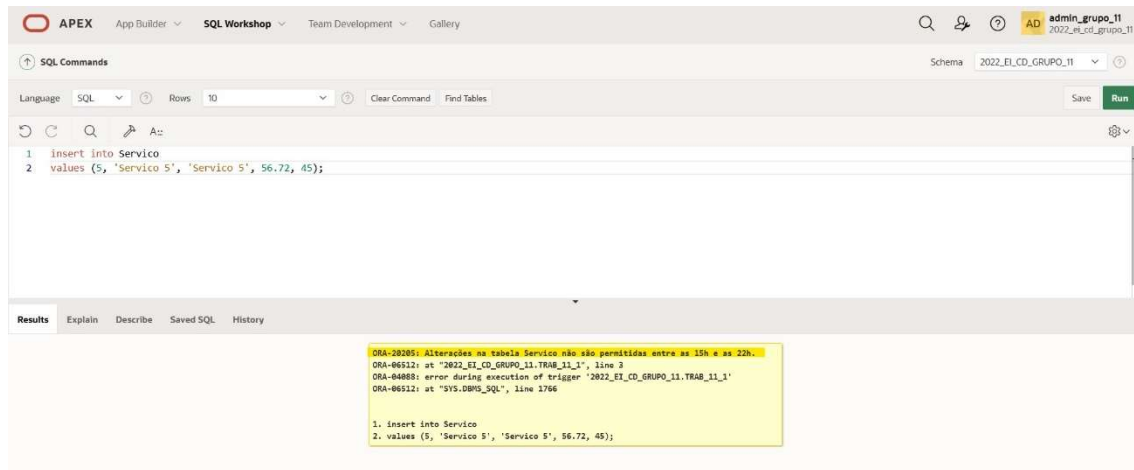
The screenshot shows the APEX SQL Workshop interface. The top navigation bar includes 'APEX', 'App Builder', 'SQL Workshop', 'Team Development', and 'Gallery'. The user is logged in as 'admin_grupo_11' with the role '2022_EL_CD_grupo_11'. The 'SQL Commands' tab is active, showing a PL/SQL script. The script includes a trigger definition and an update statement. The 'Results' tab is also visible, showing the output of the execution.

SQL Commands

```
12 http.p('Custo anterior: ' || :old.custo || '€');
13 http.p('Custo novo: ' || :new.custo || '€');
14 end if;
15 end;
16
17
18
19 update Servico
20 set custo = custo + 7
21 where id_servico = 2;
22
```

Results

Servico nº: 2
Custo anterior: 87,95€
Custo novo: 94,95€
1 row(s) updated.
0.00 seconds



TRAB_11_2

Código PLSQL

```
create or replace trigger TRAB_11_2
after update or delete or insert on Funcionario
for each row
declare
    new_audit_id number(2);
    transacao varchar2(20);
begin
    select sequencia_id_audit.nextval
    into new_audit_id
    from dual;

    transacao := case
        when UPDATING then 'UPDATE'
        when DELETING then 'DELETE'
        when INSERTING then 'INSERT'
    end;
    insert into Auditoria (id_audit, nome_tabela, nome_transacao, utilizador, data_transacao)
    values (new_audit_id, 'Funcionario', transacao, user, sysdate);
end TRAB_11_2;
```

Output

The screenshot shows the APEX SQL Workshop interface. The top navigation bar includes 'APEX', 'App Builder', 'SQL Workshop', 'Team Development', and 'Gallery'. The user is logged in as 'admin_grupo_11' with the session '2022_et_cd_grupo_11'. The 'SQL Commands' section shows a query: `select * from Auditoria;`. The 'Results' tab displays a table with 4 rows and 5 columns: ID_AUDIT, NOME_TABELA, NOME_TRANSACAO, UTILIZADOR, and DATA_TRANSACAO.

ID_AUDIT	NOME_TABELA	NOME_TRANSACAO	UTILIZADOR	DATA_TRANSACAO
1	Funcionario	UPDATE	APEX_PUBLIC_USER	05/20/2022
2	Funcionario	UPDATE	APEX_PUBLIC_USER	05/20/2022
3	Funcionario	DELETE	APEX_PUBLIC_USER	05/20/2022
4	Funcionario	INSERT	APEX_PUBLIC_USER	05/20/2022

4 rows returned in 0.00 seconds [Download](#)

TRAB_11_3

Código PLSQL

```
create or replace trigger TRAB_11_3
after update or delete or insert on Cliente
for each row
declare
    new_log_id number(2);
    transacao varchar2(20);
begin
    select sequencia_id_log.nextval
    into new_log_id
    from dual;

    transacao := case
        when UPDATING then 'UPDATE'
        when DELETING then 'DELETE'
        when INSERTING then 'INSERT'
    end;

    insert into Logs (id_log, nome_tabela, nome_transacao, utilizador,
data_transacao)
        values (new_log_id , 'Cliente', transacao, user, sysdate);
end TRAB_11_3;
```

Output

APEX

App Builder

SQL Workshop

Team Development

Gallery

admin_grupo_11

2022_el_cd_grupo_11

SQL Commands

Schema2022_EL_CD_GRUPO_11

LanguageSQL

Rows10

Clear Command

Find Tables

Save

Run

SQL Editor

```
1 select *
2 from Logs;
```

Results

Explain

Describe

Saved SQL

History

ID_LOG	NOME_TABELA	NOME_TRANSACAO	UTILIZADOR	DATA_TRANSACAO
1	Cliente	INSERT	APEX_PUBLIC_USER	05/20/2022
2	Cliente	DELETE	APEX_PUBLIC_USER	05/20/2022

2 rows returned in 0.00 seconds

Download

APEX(também 12_1 linha A)

≡ trabalho

TRAB_11.1

Inserir Novo Produto.

Insira o nome do produto:
teste

Insira o id do produto:
99

Insira a data de validade do produto:
12/12/2022

Insira a designação do produto:
teste

Selecione o serviço associado ao produto:
Servico 1

Selecione fornecedor do produto:
Fornecedor 2

Inserir Novo Produto

O produto foi inserido com sucesso.

Home

App 117

Page 34

Packages

TRAB_12_1

Código PLSQL

Construção do package

```
create or replace package TRAB_12_1 as
  procedure Inserir_Produto (nome_pr Produto.nome%type, id_pr Produto.id_produto%type,
    data_val Produto.data_validade%type, designacao_pr Produto.designacao%type, id_serv
    Produto.id_servico%type, nif_for Produto.nif_fornecedor%type);
  procedure Remover_Produto (id_prod Produto.id_produto%type);
  function Nome_Produto (id_producao Produto.id_produto%type) return varchar2;

end TRAB_12_1;
```

Criação do corpo

```
create or replace package body TRAB_12_1 as
  procedure Inserir_Produto (nome_pr Produto.nome%type, id_pr Produto.id_produto%type,
    data_val Produto.data_validade%type, designacao_pr Produto.designacao%type, id_serv
    Produto.id_servico%type, nif_for Produto.nif_fornecedor%type) as
  begin
    if nome_pr is null then
      http.p('O nome do produto não pode ser nulo.');

```
 elsif id_pr is null then
 http.p('O id do produto não pode ser nulo.');
```



```
 elsif data_val is null then
 http.p('A data de validade do produto não pode ser nulo.');
```



```
 elsif designacao_pr is null then
 http.p('A designação do produto não pode ser nulo.');
```



```
 elsif id_serv is null then
 http.p('O id de serviço do produto não pode ser nulo.');
```



```
 elsif nif_for is null then
 http.p('O nif do fornecedor do produto não pode ser nulo.');
```



```
 else
 insert into Produto (nome, id_produto, data_validade, designacao, id_servico,
nif_fornecedor)
 values (nome_pr, id_pr, data_val, designacao_pr, id_serv, nif_for);

 http.p('O produto foi inserido com sucesso.');
```



```
 end if;
 exception
 when DUP_VAL_ON_INDEX then
```


```



```

        http.p('Já existe um produto com o id ' || to_char(id_pr) || '. Por favor, indique um id
que ainda não esteja a ser usado.');
```

when others then

```

        http.p('Erro imprevisto.');
```

end Inserir_Produto;

```

procedure Remover_Produto (id_prod Produto.id_produto%type) as
reg_prod Produto%rowtype;
CHILD_RECORD_FOUND EXCEPTION;
PRAGMA EXCEPTION_INIT(CHILD_RECORD_FOUND, -02292);

begin
    if id_prod is null then
        http.p('O id do produto não pode ser nulo.');
```

else

```

        select * into reg_prod
        from Produto
        where id_produto = id_prod;

        delete from Produto
        where id_produto = id_prod;
        http.p('O Produto ' || to_char(id_prod) || ' foi removido com sucesso.');
```

end if;

exception

```

    when NO_DATA_FOUND then
        http.p('O Produto ' || to_char(id_prod) || ' não existe.');
```

when CHILD_RECORD_FOUND then

```

        http.p('O Produto ' || to_char(id_prod) || ' não pode ser removido, pois está associado a
outra tabela');
```

when others then

```

        http.p('Erro imprevisto.');
```

end Remover_Produto;

```

function Nome_Produto(id_producao Produto.id_produto%type) return varchar2 is
    nome_pr Produto.nome%type;
    n_pr Produto.id_produto%type;

begin
    n_pr := id_producao;

    select nome into nome_pr
```

```

from Produto
where id_produto = n_pr;

return(nome_pr);

end Nome_Produto;
end TRAB_12_1;

```

Output

The image displays three sequential screenshots of the APEX SQL Workshop interface, showing the execution of SQL commands. The interface includes a top navigation bar with 'APEX', 'App Builder', 'SQL Workshop', 'Team Development', and 'Gallery'. The 'SQL Commands' tab is active, and the schema is set to '2022_EI_CD_GRUPO_11'.

First Screenshot: The SQL command is a PL/SQL block that inserts a product into the 'TRAB_12_1.Inserir_Produto' table. The command is:


```

1 begin
2   TRAB_12_1.Inserir_Produto (:nome_pr, :id_pr, :data_val, :designacao_pr, :id_serv, :nif_for);
3 end;
    
```

 The results show: '0 produto foi inserido com sucesso.', 'Statement processed.', and '0.00 seconds'.

Second Screenshot: The SQL command is a PL/SQL block that removes a product from the 'TRAB_12_1.Remover_Produto' table. The command is:


```

1 begin
2   TRAB_12_1.Remover_Produto (:id_prod);
3 end;
    
```

 The results show: '0 Produto 99 foi removido com sucesso.', 'Statement processed.', and '0.02 seconds'.

Third Screenshot: The SQL command is a PL/SQL block that declares a variable and calls a procedure. The command is:


```

1 declare
2   n_produto number;
3
4 begin
5   n_produto := :id_produto;
6   http.p('Nome do produto: ' || TRAB_12_1.Nome_Produto(:id_produto));
7 end;
8
    
```

 The results show: 'Nome do produto: Produto 2.', 'Statement processed.', and '0.00 seconds'.

TRAB_12.1_b

Remover um Produto.

Selecione o produto a remover:

teste

Remover Produto

RESULTADOS

O Produto 99 foi removido com sucesso.

TRAB_13

Código PLSQL

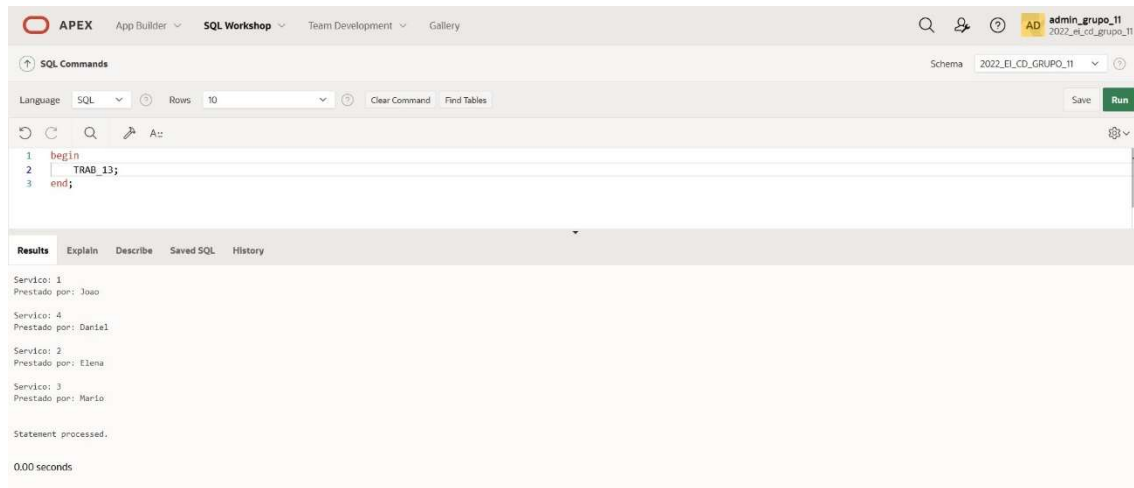
```
create or replace procedure TRAB_13 as
reg_id Presta.id_servico%type;
reg_nome Funcionario.nome%type;
cursor cursor_1 is
select id_servico, nome
from Presta, Funcionario
where Presta.id_funcionario = Funcionario.id_funcionario;
begin
  open cursor_1;
  loop
    fetch cursor_1 into reg_id, reg_nome;
    exit when cursor_1%notfound;
```

```
http.p('Servico: ' || reg_id || '<br />');  
http.p('Prestado por: ' || reg_nome || '<br />');  
http.p(' ' || '<br />');
```

```
end loop;  
close cursor_1;  
exception  
when OTHERS then  
    http.p('!Erro Imprevisto!' || '<br />');
```

```
end TRAB_13;
```

Output



The screenshot displays the APEX SQL Workshop interface. The top navigation bar includes 'APEX', 'App Builder', 'SQL Workshop', 'Team Development', and 'Gallery'. The 'SQL Commands' window shows a PL/SQL block with three lines: 'begin', 'TRAB_13;', and 'end;'. The 'Results' window shows the output of the execution, which is a series of HTML-formatted text blocks. The first block shows 'Servico: 1' and 'Prestado por: Joao'. The second block shows 'Servico: 4' and 'Prestado por: Daria1'. The third block shows 'Servico: 2' and 'Prestado por: Elena'. The fourth block shows 'Servico: 3' and 'Prestado por: Mario'. The final line of the output is 'Statement processed.' followed by '0.00 seconds'.

Apex

TRAB_13

Apresenta a lista de serviços e os funcionarios que prestam esses serviços.

RESULTADOS

Servico: 1

Prestado por: Joao

Servico: 4

Prestado por: Daniel

Servico: 2

Prestado por: Elena

Servico: 3

Prestado por: Mario

Conclusão

Mediante a realização deste trabalho foi o presente Relatório contendo escrito as etapas seguidas para a realização do mesmo desde as alterações feitas do ultimo trabalho até ao segundo como no caso modelo Relacional até a aplicação (TP2) no servidor , deparamos com certas dificuldades na elaboração do Trabalho principalmente devido ao tempo para debatermos do mesmo visto que tivemos conflitos de horários para fazê-lo, Alguns erros persistente durante a elaboração dos procedimento e triggers. Numa perspetiva global, podemos dizer que melhoramos muito desde o 1º trabalho até aqui, foi possível concluir a maioria das tarefas tidas em conta, como refazer o que achamos que era necessário, elaborar a aplicação e por fim elaboração do presente relatório.